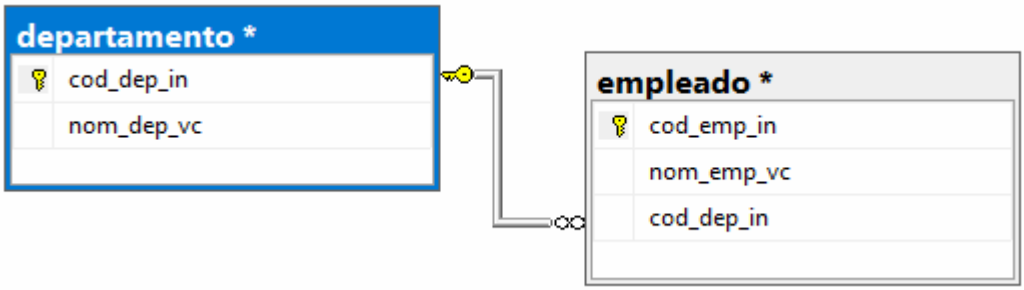
JOINs

Los JOINs en SQL Server son una forma de combinar filas de dos o más tablas basadas en una relación entre ellas.

Permiten realizar consultas que extraen datos relacionados de múltiples tablas.

```
USE master
GO
IF EXISTS
(
    SELECT *
    FROM sysdatabases
    WHERE name = 'ConsultasAvanzadas'
)
BEGIN
    DROP DATABASE ConsultasAvanzadas
END
GO
CREATE DATABASE ConsultasAvanzadas
GO
USE ConsultasAvanzadas
GO
CREATE TABLE departamento
(
    cod_dep_in INT NOT NULL,
    nom_dep_vc VARCHAR(20) NOT NULL
)
GO
CREATE TABLE empleado
(
    cod_emp_in INT NOT NULL,
    nom_emp_vc VARCHAR(20) NOT NULL,
    cod_dep_in INT NOT NULL
)
GO
INSERT INTO departamento (cod_dep_in, nom_dep_vc) VALUES
(1, 'Ventas'),
(2, 'Almacén'),
```

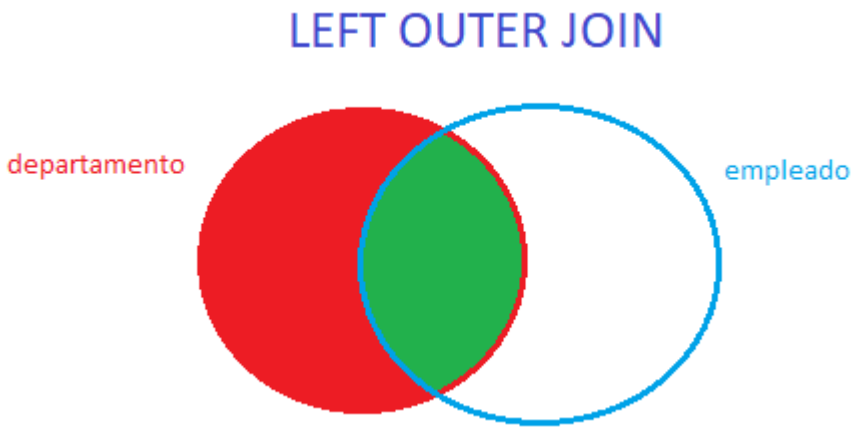
```
(3, 'Facturación'),
(4, 'Cobranzas'),
(5, 'Compras'),
(6, 'Marketing'),
(7, 'Recursos Humanos'),
(8, 'Sistemas'),
(9, 'Transportes')
GO
SELECT * FROM departamento
GO
INSERT INTO empleado (cod_emp_in, nom_emp_vc, cod_dep_in) VALUES
(1, 'Ana', 1),
(2, 'Luis', 1),
(3, 'María', 1),
(4, 'Carlos', 2),
(5, 'Sofía', 1),
(6, 'Javier', 2),
(7, 'Lucía', 3),
(8, 'Miguel', 3),
(9, 'Elena', 4),
(10, 'Pedro', 99),
(11, 'Carmen', 99),
(12, 'Diego', 99)
GO
SELECT * FROM empleado
GO
```



LEFT JOIN: Combinación externa izquierda

El **LEFT OUTER JOIN** en SQL Server es una operación de combinación que se utiliza para recuperar registros de dos tablas, mostrando todos los registros de la tabla de la izquierda (la primera tabla especificada) que no se combinan con la tabla de la derecha y los registros correspondientes de la tabla de la derecha (la segunda tabla especificada) que se encuentran combinadas con la tabla de la izquierda.

Si no hay una coincidencia en la tabla de la derecha, las columnas correspondientes de esa tabla se llenarán con valores nulos.




```
USE ConsultasAvanzadas
GO
-- LEFT OUTER JOIN para listar todos los departamentos y sus empleados,
-- incluyendo departamentos sin empleados
SELECT *
FROM departamento AS d
LEFT OUTER JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
GO
```

```
USE ConsultasAvanzadas
GO
-- LEFT JOIN para listar todos los departamentos y sus empleados,
-- incluyendo departamentos sin empleados
SELECT *
FROM departamento AS d
LEFT JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
GO
```

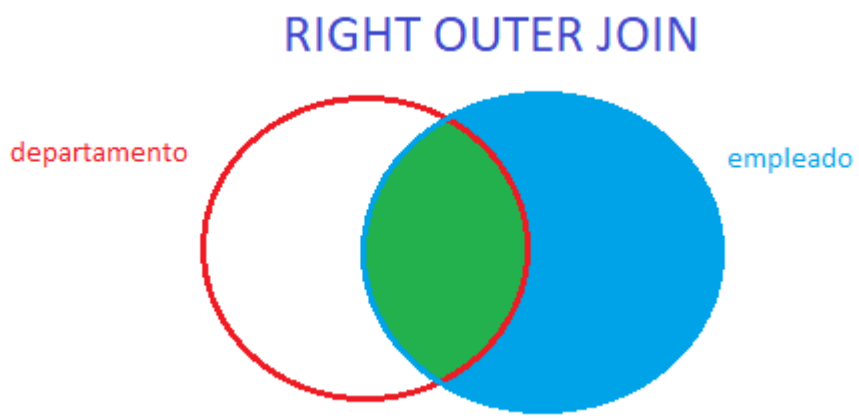
	cod_dep_in	nom_dep_vc	cod_emp_in	nom_emp_vc	cod_dep_in
1	1	Ventas	1	Ana	1
2	1	Ventas	2	Luis	1
3	1	Ventas	3	María	1
4	1	Ventas	5	Sofía	1
5	2	Almacén	4	Carlos	2
6	2	Almacén	6	Javier	2
7	3	Facturación	7	Lucía	3
8	3	Facturación	8	Miguel	3
9	4	Cobranzas	9	Elena	4
10	5	Compras	NULL	NULL	NULL
11	6	Marketing	NULL	NULL	NULL
12	7	Recursos Humanos	NULL	NULL	NULL
13	8	Sistemas	NULL	NULL	NULL
14	9	Transportes	NULL	NULL	NULL

RIGHT JOIN: Combinación externa derecha

El RIGHT OUTER JOIN en SQL Server es una operación de combinación que se utiliza para recuperar registros de dos tablas.

A diferencia del LEFT OUTER JOIN, el RIGHT OUTER JOIN muestra todos los registros de la tabla de la derecha (la segunda tabla especificada) y los registros correspondientes de la tabla de la izquierda (la primera tabla especificada).

Si no hay una coincidencia en la tabla de la izquierda, las columnas correspondientes de esa tabla se rellenarán con valores nulos.




```
USE ConsultasAvanzadas
GO
-- RIGHT OUTER JOIN entre las tablas departamento y empleado
-- mostrando todos los empleados y los departamentos correspondientes (si los hay)
SELECT *
FROM departamento AS d
RIGHT OUTER JOIN empleado AS e
```

```
ON d.cod_dep_in = e.cod_dep_in
GO
```

```
USE ConsultasAvanzadas
GO
-- RIGHT JOIN entre las tablas departamento y empleado
-- mostrando todos los empleados y los departamentos correspondientes (si los hay)
SELECT *
FROM departamento AS d
RIGHT JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
GO
```

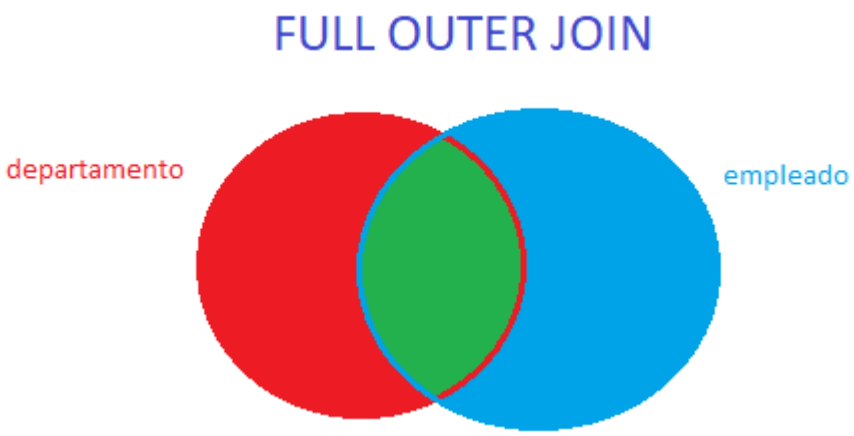
	cod_dep_in	nom_dep_vc	cod_emp_in	nom_emp_vc	cod_dep_in
1	1	Ventas	1	Ana	1
2	1	Ventas	2	Luis	1
3	1	Ventas	3	María	1
4	2	Almacén	4	Carlos	2
5	1	Ventas	5	Sofía	1
6	2	Almacén	6	Javier	2
7	3	Facturación	7	Lucía	3
8	3	Facturación	8	Miguel	3
9	4	Cobranzas	9	Elena	4
10	NULL	NULL	10	Pedro	99
11	NULL	NULL	11	Carmen	99
12	NULL	NULL	12	Diego	99

FULL JOIN: Combinación externa completa

El **FULL OUTER JOIN** en SQL Server es una operación de combinación que se utiliza para recuperar registros de dos tablas, mostrando todos los registros de ambas tablas.

Esto significa que se incluyen todas las filas de la tabla de la izquierda (la primera tabla) y todas las filas de la tabla de la derecha (la segunda tabla).

Cuando un registro de una tabla no tiene una coincidencia en la otra, las columnas de la tabla sin coincidencias se rellenarán con valores nulos.




```
USE ConsultasAvanzadas
GO
-- FULL OUTER JOIN entre departamento y empleado
-- que muestra todos los departamentos y todos los empleados,
-- incluyendo los departamentos sin empleados y los empleados sin departamento asignado.
SELECT *
FROM departamento AS d
FULL OUTER JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
GO
```

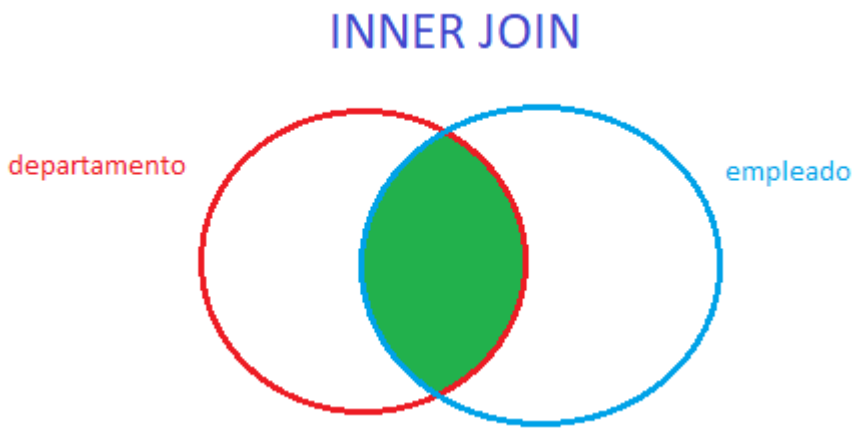
```
USE ConsultasAvanzadas
GO
-- FULL JOIN entre departamento y empleado
-- que muestra todos los departamentos y todos los empleados,
-- incluyendo los departamentos sin empleados y los empleados sin departamento asignado.
SELECT *
FROM departamento AS d
FULL JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
GO
```

	cod_dep_in	nom_dep_vc	cod_emp_in	nom_emp_vc	cod_dep_in
1	1	Ventas	1	Ana	1
2	1	Ventas	2	Luis	1
3	1	Ventas	3	María	1
4	1	Ventas	5	Sofía	1
5	2	Almacén	4	Carlos	2
6	2	Almacén	6	Javier	2
7	3	Facturación	7	Lucía	3
8	3	Facturación	8	Miguel	3
9	4	Cobranzas	9	Elena	4
10	5	Compras	NULL	NULL	NULL
11	6	Marketing	NULL	NULL	NULL
12	7	Recursos Humanos	NULL	NULL	NULL
13	8	Sistemas	NULL	NULL	NULL
14	9	Transportes	NULL	NULL	NULL
15	NULL	NULL	10	Pedro	99
16	NULL	NULL	11	Carmen	99
17	NULL	NULL	12	Diego	99

INNER JOIN: Combinación interna

El INNER JOIN en SQL Server es una operación de combinación que se utiliza para recuperar registros que tienen coincidencias en ambas tablas involucradas en la consulta.

Es uno de los tipos de uniones más comunes y se utiliza para vincular filas de dos o más tablas basadas en una condición relacionada.




```
USE ConsultasAvanzadas
GO
-- INNER JOIN entre departamento y empleado
-- para obtener los departamentos con sus empleados asignados
SELECT *
FROM departamento AS d
INNER JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
```

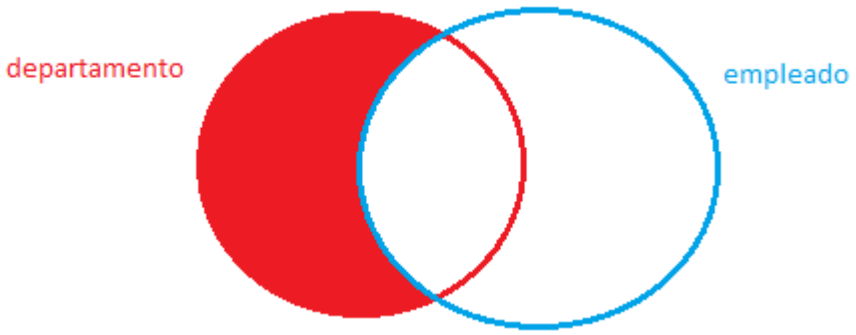
```
ORDER BY d.cod_dep_in, e.cod_emp_in
GO
```

	cod_dep_in	nom_dep_vc	cod_emp_in	nom_emp_vc	cod_dep_in
1	1	Ventas	1	Ana	1
2	1	Ventas	2	Luis	1
3	1	Ventas	3	María	1
4	1	Ventas	5	Sofía	1
5	2	Almacén	4	Carlos	2
6	2	Almacén	6	Javier	2
7	3	Facturación	7	Lucía	3
8	3	Facturación	8	Miguel	3
9	4	Cobranzas	9	Elena	4

Anticombinación Izquierda

La anticombinación izquierda se utiliza para obtener filas de una tabla que no tienen coincidencias en otra tabla.

En otras palabras, se busca todos los registros de la tabla de la izquierda que no están presentes en la tabla de la derecha.



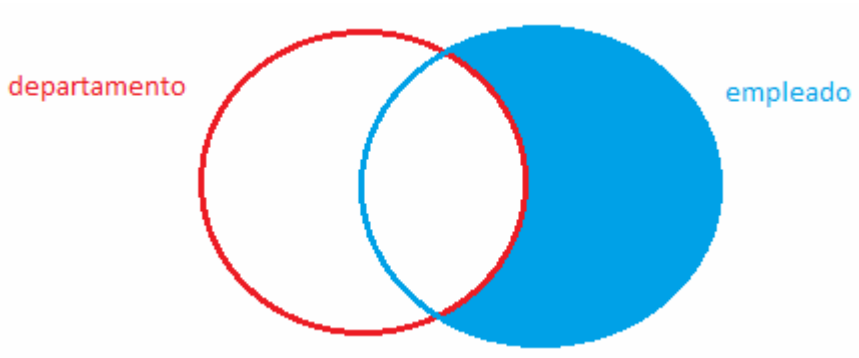

```
USE ConsultasAvanzadas
GO
-- Anticombinación izquierda entre departamento y empleado
-- Mostrar todos los departamentos que no tienen empleados asignados.
SELECT d.*
FROM departamento AS d
LEFT JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
WHERE cod_emp_in IS NULL
ORDER BY d.cod_dep_in, e.cod_emp_in
GO
```

	cod_dep_in	nom_dep_vc
1	5	Compras
2	6	Marketing
3	7	Recursos Humanos
4	8	Sistemas
5	9	Transportes

Anticombinación Derecha

La anticombinación derecha (o RIGHT ANTI JOIN) es un concepto similar a la anticombinación izquierda, pero se centra en encontrar registros en la tabla derecha que no tienen coincidencias en la tabla izquierda.

Aunque no hay una sintaxis formal para esta operación en SQL Server, se puede lograr utilizando un RIGHT JOIN combinado con una cláusula WHERE.



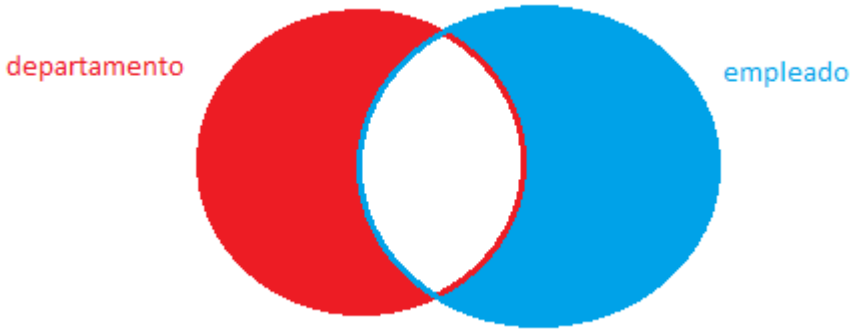

```
USE ConsultasAvanzadas
GO
-- Anticombinación derecha entre departamento y empleado
-- Mostrar todos los empleados que no tienen departamentos asignados.
SELECT e.*
FROM departamento AS d
RIGHT JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
WHERE d.cod_dep_in IS NULL
ORDER BY d.cod_dep_in, e.cod_emp_in
GO
```

	cod_emp_in	nom_emp_vc	cod_dep_in
1	10	Pedro	99
2	11	Carmen	99
3	12	Diego	99

Anticombinación Completa

La anticombinación completa permite obtener todos los registros de ambas tablas que no tienen correspondencias.

Es útil cuando deseas identificar qué registros están presentes en una tabla pero no en la otra, y viceversa.




```
USE ConsultasAvanzadas
GO
-- Anticombinación completa entre departamento y empleado
-- Empleados que no pertenecen a ningún departamento registrado
-- Departamentos que no pertenecen a ningún empleado registrado
SELECT *
FROM departamento AS d
FULL JOIN empleado AS e
ON d.cod_dep_in = e.cod_dep_in
WHERE d.cod_dep_in IS NULL OR e.cod_emp_in IS NULL
ORDER BY d.cod_dep_in, e.cod_emp_in
GO
```

	cod_dep_in	nom_dep_vc	cod_emp_in	nom_emp_vc	cod_dep_in
1	NULL	NULL	10	Pedro	99
2	NULL	NULL	11	Carmen	99
3	NULL	NULL	12	Diego	99
4	5	Compras	NULL	NULL	NULL
5	6	Marketing	NULL	NULL	NULL
6	7	Recursos Humanos	NULL	NULL	NULL
7	8	Sistemas	NULL	NULL	NULL
8	9	Transportes	NULL	NULL	NULL

CROSS JOIN

El CROSS JOIN en SQL Server es una operación de combinación que produce el producto cartesiano de dos tablas.

Esto significa que cada fila de la primera tabla se empareja con cada fila de la segunda tabla.

Como resultado, el número total de filas en el resultado es el producto del número de filas en las tablas involucradas.

```
USE ConsultasAvanzadas
GO
SELECT *
FROM departamento d
CROSS JOIN empleado e
GO
```

```
cod_dep_in  nom_dep_vc           cod_emp_in  nom_emp_vc           cod_dep_in
----------- -------------------- ----------- -------------------- -----------
1           Ventas               1           Ana                  1
1           Ventas               2           Luis                 1
1           Ventas               3           María                1
1           Ventas               4           Carlos               2
1           Ventas               5           Sofía                1
1           Ventas               6           Javier               2
1           Ventas               7           Lucía                3
1           Ventas               8           Miguel               3
1           Ventas               9           Elena                4
1           Ventas               10          Pedro                99
1           Ventas               11          Carmen               99
1           Ventas               12          Diego                99
2           Almacén              1           Ana                  1
2           Almacén              2           Luis                 1
2           Almacén              3           María                1
2           Almacén              4           Carlos               2
2           Almacén              5           Sofía                1
2           Almacén              6           Javier               2
2           Almacén              7           Lucía                3
2           Almacén              8           Miguel               3
2           Almacén              9           Elena                4
2           Almacén              10          Pedro                99
2           Almacén              11          Carmen               99
2           Almacén              12          Diego                99
3           Facturación          1           Ana                  1
3           Facturación          2           Luis                 1
3           Facturación          3           María                1
3           Facturación          4           Carlos               2
3           Facturación          5           Sofía                1
```

3	Facturación	6	Javier	2
3	Facturación	7	Lucía	3
3	Facturación	8	Miguel	3
3	Facturación	9	Elena	4
3	Facturación	10	Pedro	99
3	Facturación	11	Carmen	99
3	Facturación	12	Diego	99
4	Cobranzas	1	Ana	1
4	Cobranzas	2	Luis	1
4	Cobranzas	3	María	1
4	Cobranzas	4	Carlos	2
4	Cobranzas	5	Sofía	1
4	Cobranzas	6	Javier	2
4	Cobranzas	7	Lucía	3
4	Cobranzas	8	Miguel	3
4	Cobranzas	9	Elena	4
4	Cobranzas	10	Pedro	99
4	Cobranzas	11	Carmen	99
4	Cobranzas	12	Diego	99
5	Compras	1	Ana	1
5	Compras	2	Luis	1
5	Compras	3	María	1
5	Compras	4	Carlos	2
5	Compras	5	Sofía	1
5	Compras	6	Javier	2
5	Compras	7	Lucía	3
5	Compras	8	Miguel	3
5	Compras	9	Elena	4
5	Compras	10	Pedro	99
5	Compras	11	Carmen	99
5	Compras	12	Diego	99
6	Marketing	1	Ana	1
6	Marketing	2	Luis	1
6	Marketing	3	María	1
6	Marketing	4	Carlos	2
6	Marketing	5	Sofía	1
6	Marketing	6	Javier	2
6	Marketing	7	Lucía	3
6	Marketing	8	Miguel	3
6	Marketing	9	Elena	4
6	Marketing	10	Pedro	99
6	Marketing	11	Carmen	99
6	Marketing	12	Diego	99
7	Recursos Humanos	1	Ana	1
7	Recursos Humanos	2	Luis	1
7	Recursos Humanos	3	María	1
7	Recursos Humanos	4	Carlos	2
7	Recursos Humanos	5	Sofía	1

```
7           Recursos Humanos     6           Javier              2
7           Recursos Humanos     7           Lucía               3
7           Recursos Humanos     8           Miguel              3
7           Recursos Humanos     9           Elena               4
7           Recursos Humanos     10          Pedro               99
7           Recursos Humanos     11          Carmen              99
7           Recursos Humanos     12          Diego               99
8           Sistemas             1           Ana                 1
8           Sistemas             2           Luis                1
8           Sistemas             3           María               1
8           Sistemas             4           Carlos              2
8           Sistemas             5           Sofía               1
8           Sistemas             6           Javier              2
8           Sistemas             7           Lucía               3
8           Sistemas             8           Miguel              3
8           Sistemas             9           Elena               4
8           Sistemas             10          Pedro               99
8           Sistemas             11          Carmen              99
8           Sistemas             12          Diego               99
9           Transportes          1           Ana                 1
9           Transportes          2           Luis                1
9           Transportes          3           María               1
9           Transportes          4           Carlos              2
9           Transportes          5           Sofía               1
9           Transportes          6           Javier              2
9           Transportes          7           Lucía               3
9           Transportes          8           Miguel              3
9           Transportes          9           Elena               4
9           Transportes          10          Pedro               99
9           Transportes          11          Carmen              99
9           Transportes          12          Diego               99
```

AGREGACIÓN

La agregación en SQL Server se refiere al proceso de recopilar y resumir datos de acuerdo con criterios específicos.

Se utiliza comúnmente para calcular estadísticas como sumas, promedios, conteos y valores máximos y mínimos a partir de un conjunto de registros.

Las funciones de agregación son fundamentales para el análisis de datos en bases de datos y son ampliamente utilizadas en consultas SQL.

Funciones de Agregación

- **SUM()**: Calcula la suma de un conjunto de valores.
- **AVG()**: Calcula el promedio de un conjunto de valores.
- **COUNT()**: Cuenta el número de filas que cumplen con determinadas condiciones. Puede contar todas las filas o un número específico de columnas, donde los valores nulos son excluidos.
- **MAX()**: Devuelve el valor máximo de un conjunto de valores.
- **MIN()**: Devuelve el valor mínimo de un conjunto de valores.

```
-- Mostrar el Total de Ventas
SELECT CONVERT(DECIMAL(19,2), SUM([LineTotal])) AS TotalVentas
FROM [AdventureWorks2025].[Sales].[SalesOrderDetail]
```

```
-- Cantidad de detalles registros de ventas
USE AdventureWorks2025;
GO
SELECT COUNT(*) AS CantidadDeRegistrosDeVentas
FROM Sales.SalesOrderDetail;
```

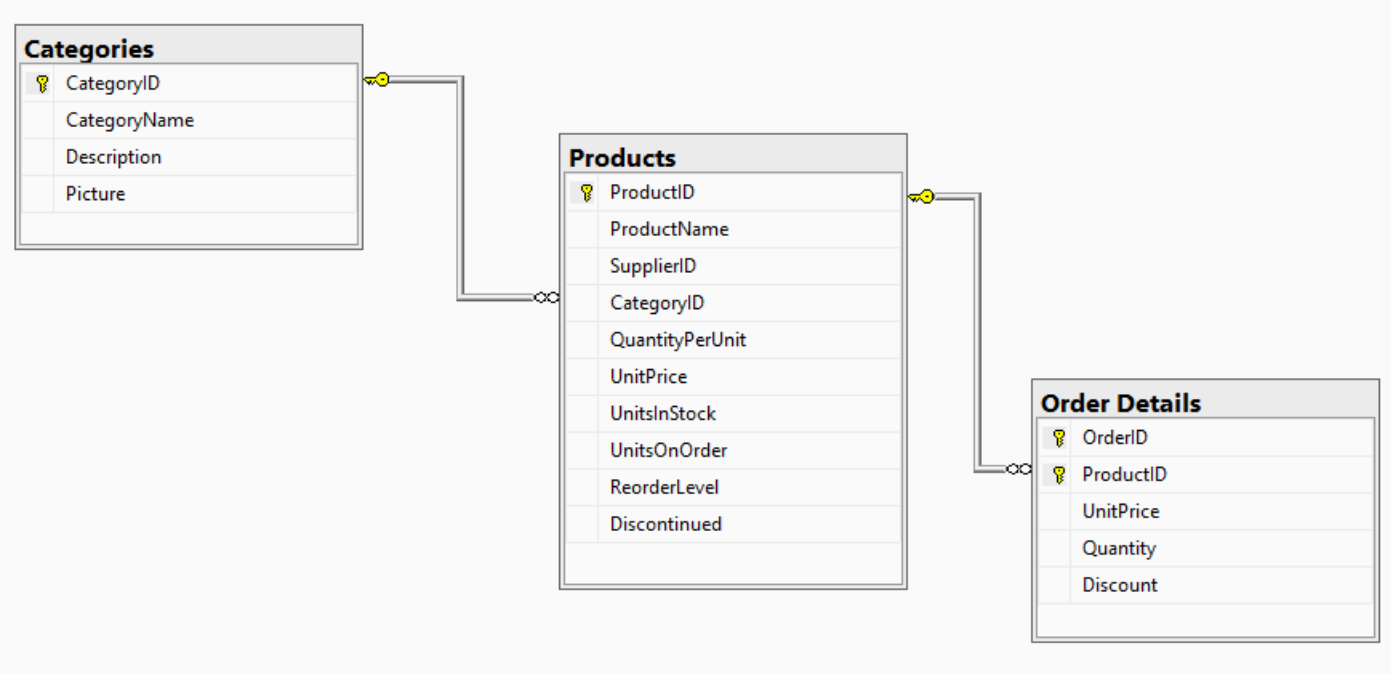
```
USE AdventureWorks2025;
GO
-- Mostrar el Total de Ventas por Internet y por las Tienda
SELECT
    SUM(CASE WHEN soh.OnlineOrderFlag = 1 THEN soh.TotalDue ELSE 0 END) AS TotalVentasInt
    SUM(CASE WHEN soh.OnlineOrderFlag = 0 THEN soh.TotalDue ELSE 0 END) AS TotalVentasTie
FROM Sales.SalesOrderHeader AS soh
```

```
USE AdventureWorks2025;
GO
-- Mostrar el Total de Ventas por Internet y por las Tienda
SELECT
    SUM(CASE WHEN soh.OnlineOrderFlag = 1 THEN soh.TotalDue ELSE 0 END) AS TotalVentasInt
    SUM(CASE WHEN soh.OnlineOrderFlag = 0 THEN soh.TotalDue ELSE 0 END) AS TotalVentasTie
FROM Sales.SalesOrderHeader AS soh

SELECT
    IIF(OnlineOrderFlag=1, 'Ventas por Internet', 'Ventas en Tienda') AS TipoVenta,
    SUM(TotalDue) AS TotalVentas
FROM Sales.SalesOrderHeader
GROUP BY OnlineOrderFlag
```

```
USE Northwind;
GO
-- Mostrar el Total de Ventas
SELECT
    CONVERT(DECIMAL(19,2), SUM(Quantity * UnitPrice * (1 - Discount))) AS [Total de Venta
FROM [Order Details]
```

Northwind: Calcular el total de ventas por Categoría




```
USE Northwind;
GO
-- Mostrar el Total de Ventas por Categoría
SELECT
    c.CategoryName AS [Categoría],
    CONVERT(DECIMAL(19,2), SUM(Quantity * od.UnitPrice * (1 - Discount))) AS [Total de Ve
FROM Categories AS c
JOIN Products AS p ON c.CategoryID = p.CategoryID
JOIN [Order Details] AS od ON p.ProductID = od.ProductID
GROUP BY c.CategoryName
ORDER BY [Total de Ventas] DESC;
```

	Categoría	Total de Ventas
1	Beverages	267868.18
2	Dairy Products	234507.29
3	Confections	167357.22
4	Meat/Poultry	163022.36
5	Seafood	131261.74
6	Condiments	106047.08
7	Produce	99984.58
8	Grains/Cereals	95744.59

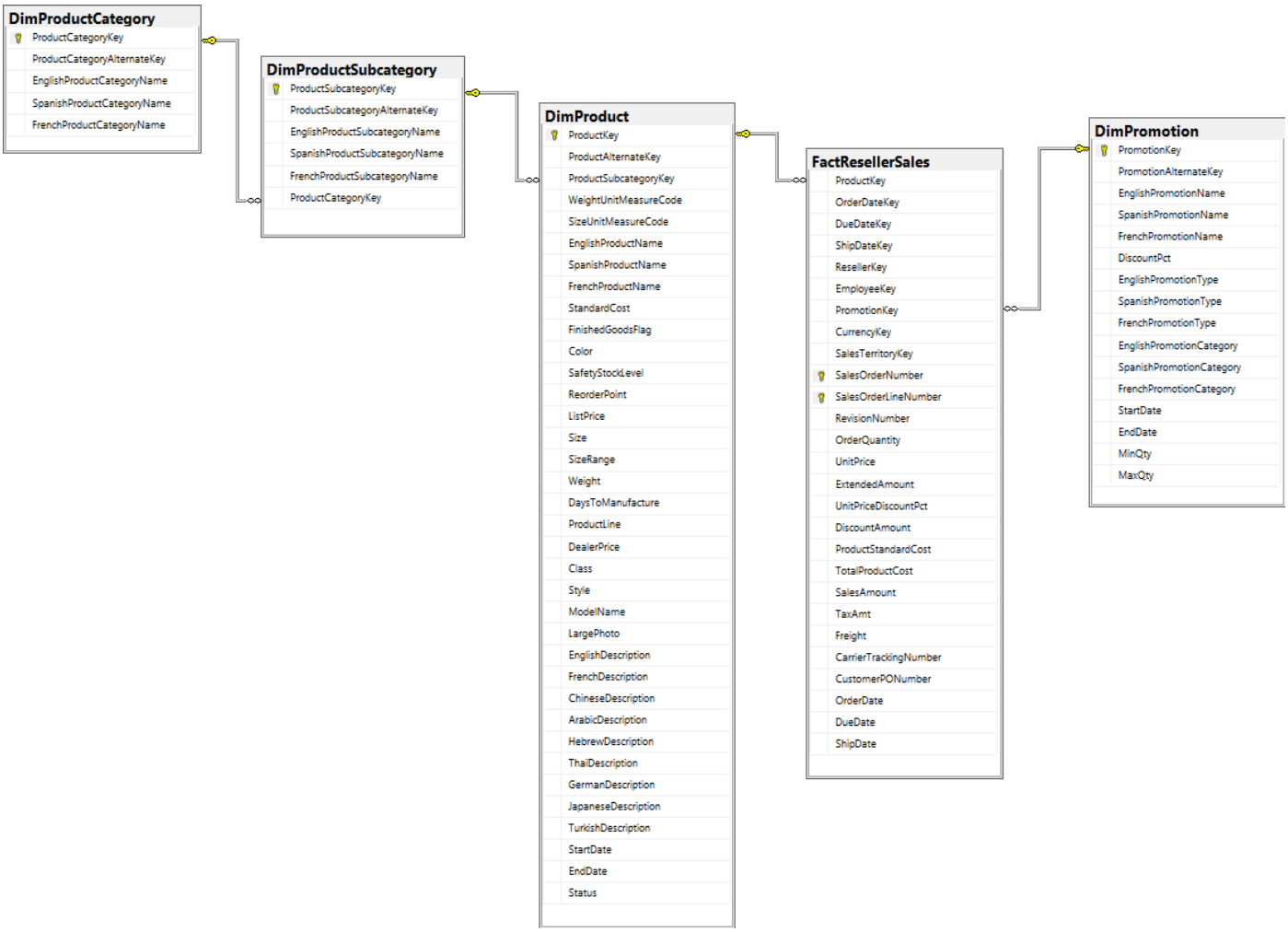
```
USE AdventureWorksDW2025;
GO
-- ==============================================
-- Ventas Totales en Reseller
SELECT
    SUM(SalesAmount) AS Microsoft,
    SUM(OrderQuantity*UnitPrice-DiscountAmount) AS TotalSales
FROM dbo.FactResellerSales;
```

	Microsoft	TotalSales
1	80450596.9823	80450596.944499

```
USE AdventureWorksDW2025;
GO
-- ==============================================
-- Ventas Totales en Reseller
SELECT
    SUM(SalesAmount) AS Microsoft,
    SUM(OrderQuantity*UnitPrice-DiscountAmount) AS Ventas,
    SUM
    (
        OrderQuantity*UnitPrice
        *
        (
            1
            -
            IIF
            (
                    f.OrderDate BETWEEN p.StartDate AND p.EndDate
                AND
                    OrderQuantity>=p.MinQty
                AND
                    OrderQuantity<=ISNULL(p.MaxQty, 999999999)
                ,
                p.DiscountPct,
```

```
                0

            )
        )
    ) AS VentasDimPromotion
FROM dbo.FactResellerSales f
INNER JOIN DimPromotion p ON f.PromotionKey = p.PromotionKey
GO
```




```
USE AdventureWorksDW2025;
GO
-- ==============================================
-- Ventas Totales por Categoria
SELECT
    pc.EnglishProductCategoryName AS Categoria,

    SUM(SalesAmount) AS Microsoft,
    SUM(OrderQuantity*UnitPrice-DiscountAmount) AS Ventas,
    SUM
    (
        OrderQuantity*UnitPrice
        *
        (
```

```
            1
            -
            IIF
            (
                    f.OrderDate BETWEEN pr.StartDate AND pr.EndDate
                AND
                    f.OrderQuantity>=pr.MinQty
                AND
                    f.OrderQuantity<=ISNULL(pr.MaxQty, 999999999)
                ,
                pr.DiscountPct,
                0

            )
        )
    ) AS VentasDimPromotion
FROM dbo.DimProductCategory pc
INNER JOIN dbo.DimProductSubcategory psc
ON pc.ProductCategoryKey = psc.ProductCategoryKey
INNER JOIN dbo.DimProduct p
ON psc.ProductSubcategoryKey = p.ProductSubcategoryKey
INNER JOIN dbo.FactResellerSales f
ON p.ProductKey = f.ProductKey
INNER JOIN DimPromotion pr ON f.PromotionKey = pr.PromotionKey
GROUP BY pc.EnglishProductCategoryName
GO
```

	Categoria	Microsoft	Ventas	VentasDimPromotion
1	Accessories	571297.9278	571297.923200002	571297.923926002
2	Bikes	66302381.557	66302381.5359969	66302381.5420283
3	Clothing	1777840.8391	1777840.82719991	1777840.83280092
4	Components	11799076.6584	11799076.6580998	11799076.6584298

[Bases de Datos Adventureworks]

SUBCONSULTAS

Las subconsultas en SQL Server son consultas anidadas dentro de otra consulta.

Permiten realizar operaciones más complejas y pueden utilizarse en diferentes partes de una consulta, como en la cláusula SELECT, FROM, WHERE, o como una expresión en conjunto.

Subconsultas Escalares (Devuelven un solo valor)

```
USE [Northwind]
GO
SELECT
    [ProductID]
    ,[ProductName]
    ,[SupplierID]
    ,(
        SELECT s.CompanyName
        FROM Suppliers s
        WHERE  SupplierID = p.[SupplierID]
    ) AS SupplierName
    ,[CategoryID]
    ,(
        SELECT CategoryName
        FROM Categories c
        WHERE  CategoryID = p.[CategoryID]
    ) AS CategoryName
    ,[QuantityPerUnit]
    ,[UnitPrice]
    ,[UnitsInStock]
    ,[UnitsOnOrder]
    ,[ReorderLevel]
    ,[Discontinued]
FROM [dbo].[Products] p
GO
```

	ProductID	ProductName	SupplierID	SupplierName	CategoryID	CategoryName	QuantityPerUnit	UnitPrice	UnitsInStock	UnitsOnOrder	ReorderLevel
1	1	Chai	1	Exotic Liquids	1	Beverages	10 boxes x 20 bags	18.00	39	0	10
2	2	Chang	1	Exotic Liquids	1	Beverages	24 - 12 oz bottles	19.00	17	40	25
3	3	Aniseed Syrup	1	Exotic Liquids	2	Condiments	12 - 550 ml bottles	10.00	13	70	25
4	4	Chef Anton's Cajun Seasoning	2	New Orleans Cajun Delights	2	Condiments	48 - 6 oz jars	22.00	53	0	0
5	5	Chef Anton's Gumbo Mix	2	New Orleans Cajun Delights	2	Condiments	36 boxes	21.35	0	0	0

```
/*
¿Cuantas veces ha sido ordenado un producto?
*/
USE Northwind;
GO
SELECT
    p.ProductID AS Código,
    p.ProductName AS Producto,
    (
        SELECT COUNT(*)
        FROM [dbo].[Order Details]
        WHERE ProductID = p.ProductID
    ) AS [Cantidad Ordenada]
FROM [dbo].[Products] p
```

```
ORDER BY [Cantidad Ordenada] DESC;
GO
```

```
/*
¿Cuantas veces ha sido ordenado un producto?
*/
USE Northwind;
GO
SELECT
    p.ProductID AS Código,
    p.ProductName AS Producto,
    (
        SELECT COUNT(*)
        FROM [Order Details]
        WHERE ProductID = p.ProductID
    ) AS [Cantidad Ordenada]
FROM Products p
ORDER BY [Cantidad Ordenada] DESC;
GO
```

	Código	Producto	Cantidad Ordenada
1	59	Raclette Courdavault	54
2	60	Camembert Pierrot	51
3	24	Guaraná Fantástica	51
4	31	Gorgonzola Telino	51
5	56	Gnocchi di nonna Alice	50
6	62	Tarte au sucre	48
7	41	Jack's New England Clam Chowder	47
8	75	Rhönbräu Klosterbier	46
9	2	Chang	44
10	16	Pavlova	43
11	71	Flotemysost	42

```
-- Comparar el precio del producto con el precio promedio
-- de todos los productos en la tabla de Productos.

USE Northwind;
GO
SELECT ProductName AS Producto,
       UnitPrice AS Precio,
       CASE
           WHEN UnitPrice > (SELECT AVG(UnitPrice) FROM dbo.Products) THEN 'Mayor al Prom
           WHEN UnitPrice < (SELECT AVG(UnitPrice) FROM dbo.Products) THEN 'Menor al Prom
           ELSE 'Promedio'
       END AS 'Comparación de Precios'
```

```
FROM dbo.Products
ORDER BY 3;
```

```
USE Northwind;
GO
DECLARE @PromedioPrecio MONEY = (SELECT AVG(UnitPrice) FROM dbo.Products);
SELECT ProductName AS Producto,
       UnitPrice AS Precio,
       CASE
           WHEN UnitPrice > @PromedioPrecio THEN 'Mayor al Promedio'
           WHEN UnitPrice < @PromedioPrecio THEN 'Menor al Promedio'
           ELSE 'Promedio'
       END AS 'Comparación de Precios'
FROM dbo.Products
ORDER BY 3;
```

```
-- Comparar el precio del producto con el precio promedio
-- de todos los productos en la tabla de Productos.

USE Northwind;
GO
DECLARE @PromedioPrecio MONEY = (SELECT AVG(UnitPrice) FROM dbo.Products);
SELECT ProductName AS Producto,
       UnitPrice AS Precio,
       CASE
           WHEN UnitPrice > @PromedioPrecio THEN CONCAT('Mayor al Precio Promedio: ', @Pr
           WHEN UnitPrice < @PromedioPrecio THEN CONCAT('Menor al Precio Promedio: ', @Pr
           ELSE 'Promedio'
       END AS 'Comparación de Precios'
FROM dbo.Products
ORDER BY 3;
```

```
-- Comparar el precio del producto con el precio promedio
-- de todos los productos en la tabla de Productos.

USE Northwind;
GO
SELECT ProductName AS Producto,
       UnitPrice AS Precio,
       CASE
           WHEN UnitPrice > (SELECT AVG(UnitPrice) FROM dbo.Products) THEN CONCAT('Mayor
           WHEN UnitPrice < (SELECT AVG(UnitPrice) FROM dbo.Products) THEN CONCAT('Menor
           ELSE 'Promedio'
```

```
        END AS 'Comparación de Precios'
FROM dbo.Products
ORDER BY 3;
```

	Producto	Precio	Comparación de Precios
1	Uncle Bob's Organic Dried Pears	30.00	Mayor al Precio Promedio: $ 28.87
2	Northwoods Cranberry Sauce	40.00	Mayor al Precio Promedio: $ 28.87
3	Mishi Kobe Niku	97.00	Mayor al Precio Promedio: $ 28.87
4	Ikura	31.00	Mayor al Precio Promedio: $ 28.87
5	Queso Manchego La Pastora	38.00	Mayor al Precio Promedio: $ 28.87
6	Sir Rodney's Marmalade	81.00	Mayor al Precio Promedio: $ 28.87
7	Gumbär Gummibärchen	31.23	Mayor al Precio Promedio: $ 28.87
8	Schoggi Schokolade	43.90	Mayor al Precio Promedio: $ 28.87
9	Rössle Sauerkraut	45.60	Mayor al Precio Promedio: $ 28.87
10	Thüringer Rostbratwurst	123.79	Mayor al Precio Promedio: $ 28.87
11	Mascarpone Fabioli	32.00	Mayor al Precio Promedio: $ 28.87

Subconsultas de lista (Multivalor) en la cláusula WHERE

```
USE Northwind;
GO
/*
    Listar los Clientes que han realizado pedidos.
*/
SELECT
    CustomerID,
    CompanyName
FROM dbo.Customers
WHERE CustomerID IN
(
    SELECT DISTINCT CustomerID
    FROM Orders
);
```

```
USE Northwind;
GO
/*
    Listar los Clientes que NO han realizado pedidos.
*/
SELECT
    CustomerID,
    CompanyName
FROM dbo.Customers
WHERE CustomerID NOT IN
```

```
(
    SELECT DISTINCT CustomerID
    FROM Orders
);
```

```
/*
    Listar los productos que pertenezcan
    a la categoría de Bebidas y Condimentos
*/
USE Northwind;
GO
SELECT
    ProductID,
    ProductName,
    CategoryID
FROM Products
WHERE CategoryID IN
(
    SELECT CategoryID
    FROM Categories
    WHERE CategoryName IN ('Beverages', 'Condiments')
);
```

El poder de EXISTS y NOT EXISTS

```
/*
    Mostrar los Clientes que si tienen ordenes de compra
*/
USE Northwind;
GO
SELECT DISTINCT C.CustomerID, C.CompanyName, C.ContactName, C.Country
FROM dbo.Customers AS C
WHERE EXISTS (
    SELECT 1
    FROM dbo.Orders AS O
    WHERE O.CustomerID = C.CustomerID
);
```

```
/*
    Mostrar los Clientes que NO tienen ordenes de compra
*/
USE Northwind;
GO
```

```
SELECT DISTINCT C.CustomerID, C.CompanyName, C.ContactName, C.Country
FROM dbo.Customers AS C
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Orders AS O
    WHERE O.CustomerID = C.CustomerID
);
```

```
/*
    Mostrar los Productos que no han sido ordenados por ningún cliente.
*/
USE Northwind;
GO
SELECT *
FROM dbo.Products AS P
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.[Order Details] AS OD
    WHERE OD.ProductID = P.ProductID
);
```

```
/*
    Mostrar los Clientes con pedidos mayores a $10000
*/
USE Northwind;
GO
SELECT
    c.CustomerID,
    c.CompanyName
FROM Customers c
WHERE
EXISTS
(
    SELECT
        o.CustomerID
    FROM Orders o
    INNER JOIN [Order Details] od
    ON o.OrderID = od.OrderID
    WHERE o.CustomerID = c.CustomerID
    GROUP BY o.CustomerID
    HAVING SUM(od.UnitPrice * od.Quantity * ( 1 - od.Discount )) > 10000
);
GO
```

Operadores Cuantificadores: ANY, SOME y ALL

> - ANY y SOME: Piensa en al menos **ALGUNO**. La condición se cumple con al menos un valor.

```
/*
    Mostrar los productos cuyo precio de venta son mayores
    que cualquier producto de la categoría 'Bebidas'.
*/
SELECT ProductID, ProductName, UnitPrice
FROM Products
WHERE UnitPrice > ANY
(
    SELECT UnitPrice
    FROM Products
    WHERE CategoryID = 1
)
```

```
/*
    Mostrar los productos cuyo precio de venta son mayores
    que cualquier producto de la categoría 'Bebidas'.
*/
SELECT ProductID, ProductName, UnitPrice
FROM Products
WHERE UnitPrice > SOME
(
    SELECT UnitPrice
    FROM Products
    WHERE CategoryID = 1
)
```

> - **ALL**; Piensa en Todos - La condición de cumple con todos los valores

```
/*
    Mostrar los productos más caros que todos
    los productos de la categoría 'Condimentos'.
*/
SELECT ProductID, ProductName, UnitPrice
FROM Products
WHERE UnitPrice > ALL
(
```

```
    SELECT UnitPrice
    FROM Products
    WHERE CategoryID = 2
)
```

Subconsultas en el Origen de Datos (Tablas Derivadas)

```
/*
    Crear un reporte de ventas por Empleado
*/
USE Northwind;
GO
SELECT
    e.EmployeeID,
    e.FirstName + ' ' + e.LastName AS EmployeeName,
    sc.TotalOrdenes,
    sc.TotalVentas
FROM Employees e
INNER JOIN
(
    SELECT
        o.EmployeeID,
        COUNT(*) AS TotalOrdenes,
        CAST(SUM(od.UnitPrice * od.Quantity * (1 - od.Discount)) AS DECIMAL(19,2)) AS Tot
    FROM Orders o
    INNER JOIN [Order Details] od ON o.OrderID = od.OrderID
    GROUP BY o.EmployeeID
) AS sc
ON e.EmployeeID = sc.EmployeeID
ORDER BY sc.TotalVentas DESC;
```

COMMON TABLE EXPRESSION (CTE)

Un CTE es un conjunto de resultados temporal derivado de una consulta que puede ser utilizado como una tabla derivada. Su comportamiento es muy similar al de una vista. Una CTE puede contener referencias a ella misma, por lo que permite diseñar consultas recursivas.

Sintaxis

```
WITH nombre_CTE( lista_columnas )
AS
```

```
(
SELECT_que_puebla_la_CTE
)
SELECT_que_muestra_la_CTE
```

```
USE Northwind
GO
WITH VentasCTE AS (
    SELECT
        YEAR(o.OrderDate) AS Año,
        DATENAME(MONTH, o.OrderDate) AS Mes,
        SUM(od.Quantity * od.UnitPrice * (1 - od.Discount)) AS TotalVentas
    FROM Orders o
    INNER JOIN [Order Details] od
    ON o.OrderID = od.OrderID
    GROUP BY YEAR(o.OrderDate), DATENAME(MONTH, o.OrderDate)
)
SELECT * FROM VentasCTE
WHERE Año = 1997 AND Mes IN ('Marzo', 'Abril', 'Mayo')
GO
```

UNION

El operador UNION en SQL Server se utiliza para combinar los resultados de dos o más consultas en un solo conjunto de resultados.

Es importante seguir ciertas reglas y condiciones para que se pueda utilizar correctamente.

Características UNION

> - **Número de Columnas**: Las consultas que se combinan deben tener el mismo número de columnas en el conjunto de resultados.
> - **Tipos de Datos**: Las columnas correspondientes en cada consulta deben tener tipos de datos compatibles.
> - **Eliminación de Duplicados**: Por defecto, el operador **UNION** elimina los registros duplicados en el conjunto de resultados. Si deseas incluir duplicados, se puede usar **UNION ALL**.

```
USE [Northwind]
GO
SELECT
    'CLI' AS Tipo
    ,[CustomerID]
    ,[CompanyName]
    ,[ContactName]
    ,[ContactTitle]
    ,[Address]
    ,[City]
    ,[Region]
    ,[PostalCode]
    ,[Country]
    ,[Phone]
    ,[Fax]
FROM [dbo].[Customers]
UNION
SELECT
    'EMP' AS Tipo
    ,LTRIM([EmployeeID])
    ,CONCAT([TitleOfCourtesy], ' ', [LastName] ,', ', [FirstName])
    ,NULL
    ,[Title]

    ,[Address]
    ,[City]
    ,[Region]
    ,[PostalCode]
    ,[Country]
    ,CONCAT([HomePhone], '- ', [Extension]) AS [Phone]
    , NULL
FROM [dbo].[Employees]
UNION
SELECT
    'PRV' AS Tipo
    ,LTRIM([SupplierID])
    ,[CompanyName]
    ,[ContactName]
    ,[ContactTitle]
    ,[Address]
    ,[City]
    ,[Region]
    ,[PostalCode]
    ,[Country]
    ,[Phone]
```